some can be used as a matter of preference. What follows is a list of topics you might want to dig into:

- *Class-based views*—Throughout this book you've seen views as functions, Django also supports views based on classes. Django provides classes you can use directly or inherit from to define your own. Class-based views (CBVs) can mean less code is needed when dealing with multiple HTTP Methods, as you use one class method per action. You can learn more about CBVs here: https://mng.bz/67YG. The community is somewhat divided on the use of this feature, and personally, I prefer function-based views, as they are a little less abstract, but there are plenty of coders who prefer using CBVs. A great, in-depth article by one of the core Django developers explains why he still uses function based views most of the time: https://mng.bz/oe0N.
- *Middleware*—In the last few chapters, you've seen third-party libraries that have middleware components. Middleware is code that gets run before or after a view, allowing global actions on all requests or responses. You can write your own middleware using a factory function or a class. If you find you're writing the same snippet of code in many of your views, middleware might be a better approach. Writing middleware is covered in the Django documentation: https://mng.bz/ng0v.
- *Custom template tags and filters*—Django is structured around a few core concepts, with everything else being a pluggable component. Template tags and filters are no different. You can write your own tags and filters, and in fact there are shortcut mechanisms that make certain kinds of tags just a few lines of code. The Django Awl library, covered in chapter 13, has a few tags I've written to make my templating life easier—for example, `{% qif %}` is an inline `if` conditional, for those cases where a block takes up a lot of space. To learn how to write your own tags and filters visit the docs: https://mng.bz/v8Jx. Alternatively, if you aren't tired of my writing style yet, you can try this article: https://mng.bz/4JpB.
- *Validators*—A validator is code that checks a value and raises a `ValidationError` if it doesn't meet some criteria. Django ships with over twenty built-in validators, and some `Model` fields are simply a `CharField` with a specific validator attached. Of course, you can also write your own. To learn more, you can read about the topic here: https://mng.bz/QZVm.
- *Complex database operations*—In chapter 12, you got a quick introduction to `Q` objects, which get used to build complex ORM queries. Throughout this book you've been using `.filter()` and `.all()`, but the query language is far deeper than that. Full documentation on the API is available here: https://mng.bz/X1VY. If that's not enough, Django also supports query expressions that perform calculations in the database without pulling the object into memory. This includes performing arbitrary math on a field and building aggregates like sums. Documentation on query expressions is available here: https://mng.bz/y8oJ. A very useful cheat sheet on all things ORM is also available on the following web page: https://djangocentral.com/django-orm-cheatsheet/.